Intro to SAT Solving with Z3

Propositional connectives in the Z3 Python API (z3-solver)

Setup

We are assuming you are familiar with setting up virtual enviroments and running python libraries. If there is any difficulty, kindly refer to the readme.md provided along with this pdf file.
Install with `pip install z3-solver`. Boolean variables are declared with `Bool()` or `Bools()` before being combined with connectives.
In the end of this doc you have a demo file. All of you are expected to run this file and come before class on Thursday.

Initialisation

```
from z3 import *

p, q, r = Bools('p q r')
s = Solver()
```

AND : conjunction

Use `And(...)`. Takes two or more arguments.

```
s.add(And(p, q))          # p AND q
s.add(And(p, q, r))       # p AND q AND r  (any number of args)
```

Python's built-in `&` operator also works between two Z3 Bool expressions:

```
s.add(p & q)               # same as And(p, q)
```

OR : disjunction

Use `Or(...)`. Also takes two or more arguments.

```
s.add(Or(p, q))            # p OR q
s.add(Or(p, q, r))         # p OR q OR r
```

Equivalent shorthand with the Python `|` operator:

```
s.add(p | q)                # same as Or(p, q)
```

NOT : negation

Use `Not(...)`. Takes exactly one argument.

```
s.add(Not(p))               # NOT p
```

Equivalent shorthand with the Python `~` operator:

```
s.add(~p)                   # same as Not(p)
```

IMPLIES : implication

Use `Implies(a, b)` for "a implies b" ($a \to b$). Order matters: the first argument is the antecedent, the second is the consequent.

```
s.add(Implies(p, q))        # p -> q  (if p then q)
```

There is no infix shorthand operator for implication in Z3 — always use `Implies(...)` explicitly.

IFF : biconditional (if and only if)

For Bool sorts, plain Python `==` between two Z3 Bool expressions builds the biconditional ($p \leftrightarrow q$), not a Python equality check. This is the idiomatic way to write iff in Z3.

```
s.add(p == q)               # p <-> q  (p iff q)
```

You can also spell it out manually as a conjunction of two implications:

```
s.add(And(Implies(p, q), Implies(q, p)))   # equivalent to p == q
```

Note: use `==` for iff, but `!=` for XOR (exclusive or) or use the explicit `Xor(p, q)`.

Quick reference

Connective	Z3 function	Shorthand	Meaning
AND	`And(p, q)`	`p & q`	p and q
OR	`Or(p, q)`	`p | q`	p or q
NOT	`Not(p)`	`~p`	not p
IMPLIES	`Implies(p, q)`	—	if p then q
IFF	`p == q`	—	p if and only if q
XOR	`Xor(p, q)`	`p != q`	exactly one of p, q

Worked example: solving a small formula

Putting all five connectives together and asking Z3 to find a satisfying assignment:

TRY THIS DEMO CODE ON YOUR LOCAL MACHINES.
Make a file titled "demo.py" and run this code in your virtual environment using:

```
python3 demo.py
```

```
from z3 import *

p, q, r = Bools('p q r')
s = Solver()

s.add(And(p, q))           # p and q must both hold
s.add(Or(p, r))             # p or r must hold
s.add(Not(r))                # r must be false
s.add(Implies(p, q))       # p implies q
s.add(p == q)                 # p iff q

if s.check() == sat:
    print(s.model())
else:
    print('unsatisfiable')
```

Output: `[r = False, q = True, p = True]` the only assignment consistent with every constraint above.